



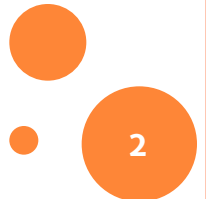
SERVICIOS Y BROADCAST



SERVICIOS

○ Service

- Componente que no necesita interacción con y del usuario (se ejecuta en segundo plano)
- Puede ser lanzado y detenido a voluntad
- No tiene interfaz gráfica
- Pensados para realizar tareas que no interfieran con el resto de aplicaciones / funciones
 - Reproductor de música
 - Geolocalización "continua"
 - Envío o descarga de ficheros
 - ...
- No es ni un proceso diferente, ni corre en un hilo diferente. Se ejecuta en el mismo proceso que la aplicación.



SERVICIOS

○ Tipos de servicios

- Primer plano (Foreground)
 - Un servicio en primer plano realiza una operación que el usuario puede notar. Por ejemplo, un reproductor de música.
 - Los servicios en primer plano deben mostrar una notificación.
 - Continúan ejecutándose incluso cuando el usuario deja de interactuar con la app.
- Segundo plano (Background)
 - Un servicio en segundo plano realiza una operación que el usuario no nota directamente. (Tema: Trabajo en Segundo Plano)
- Vinculado (Bound)
 - Un servicio está **vinculado** cuando un componente de la aplicación se vincula a él llamando a `bindService()`.
 - Ofrece una interfaz cliente-servidor que permite que los componentes interactúen con el servicio



SERVICIOS

○ Foreground Service

- Se define a través de una clase que extiende a la clase **Service**

```
public class MiServicio extends Service {  
    ...  
}
```

- Para lanzar un servicio en primer plano desde una actividad, se usa un intent y el método **startForegroundService(..)**

```
Intent i =new Intent(this,MiServicio.class);  
startForegroundService(i);
```

La clase que extiende a Service

- Deben ir declarados en el manifiesto y añadir permisos

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>  
<service android:name=".MiServicio"/>
```



SERVICIOS

- Service
 - Ciclo de vida:

startService()

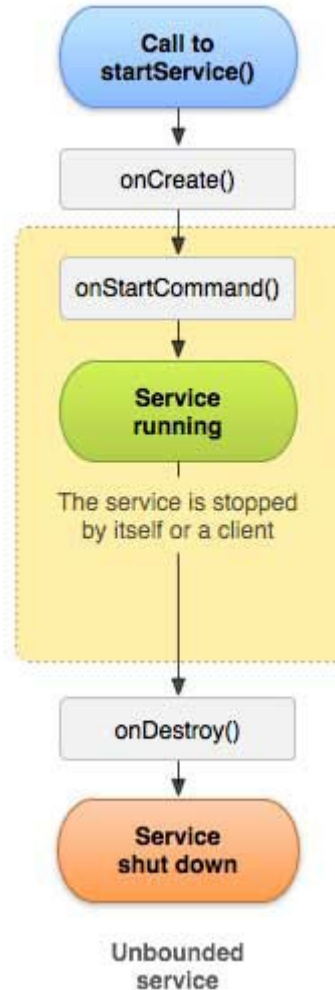


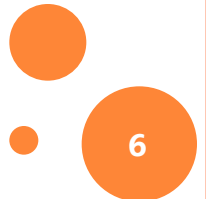
Imagen de: <https://developer.android.com/guide/components/services>



SERVICIOS

○ Service

- Su funcionamiento varía en función del SDK target
- Si el target de la app es un SDK < 26 (Oreo)
 - El servicio se puede ejecutar en segundo plano, sin que el usuario sea consciente
- Si el target de la app es un SDK ≥ 26 (Oreo)
 - El servicio sólo se ejecuta mientras :
 - La actividad que lo lanzó esté en primer plano (y un margen de varios minutos desde que pase a segundo plano)
 - Si la aplicación deja de ejecutarse, el servicio se detiene
 - Exista una indicación (notificación) en la barra de tareas indicando que el servicio se está ejecutando



SERVICIOS

○ Service

- Para evitar que el servicio se pare si el SDK del target ≥ 26
 - Se debe lanzar mediante *startForegroundService()*

```
Intent myService = new Intent(this, MyService.class);  
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
    startForegroundService(myService);  
} else {  
    startService(myService);  
}
```

Hay que hacer una condición porque *startForegroundService()* sólo funciona a partir de la API26



SERVICIOS

○ Service

- En el servicio en primer plano, se debe lanzar una notificación usando el método **startForeground()**. Se puede indicar el tipo de servicio.

La notificación estará visible mientras el servicio esté en ejecución (setAutoCancel a false). El usuario puede ocultarla desde los ajustes (no mostrar notificaciones del canal o de la app), pero no quitarla.

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
    NotificationManager elmanager = (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);  
    NotificationChannel canalservicio = new NotificationChannel("IdCanal",  
                                                                "NombreCanal", NotificationManager.IMPORTANCE_DEFAULT);  
    elmanager.createNotificationChannel(canalservicio);  
    NotificationCompat.Builder builder = new NotificationCompat.Builder(this, "IdCanal")  
        .setSmallIcon(R.drawable.small_icon)  
        .setContentTitle(getString(R.string.app_name))  
        .setAutoCancel(false);  
    Notification notification = builder.build();  
    int type = ServiceInfo.FOREGROUND_SERVICE_TYPE_SPECIAL_USE;  
    startForeground(1, notification, type);  
}
```

Se puede usar icono de Android o uno personalizado

Identificador para la notificación

Se indica el tipo de servicio en primer plano. Añadir también en el manifiesto, dentro del servicio.



SERVICIOS

○ Service

- En el servicio en primer plano, se debe lanzar una notificación usando el método **startForeground()**. Sin indicar el tipo de servicio.

La notificación estará visible mientras el servicio esté en ejecución (setAutoCancel a false). El usuario puede ocultarla desde los ajustes (no mostrar notificaciones del canal o de la app), pero no quitarla.

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
  
    NotificationManager elmanager = (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);  
    NotificationChannel canalservicio = new NotificationChannel("IdCanal",  
                                                                "NombreCanal", NotificationManager.IMPORTANCE_DEFAULT);  
    elmanager.createNotificationChannel(canalservicio);  
    NotificationCompat.Builder builder = new NotificationCompat.Builder(this, "IdCanal")  
        .setSmallIcon(R.drawable.small_icon)  
        .setContentTitle(getString(R.string.app_name))  
        .setAutoCancel(false);  
    Notification notification = builder.build();  
    startForeground(1, notification);  
}
```

Identificador para la notificación

Sin añadir mostrar el tipo de servicio



SERVICIOS

○ Service

- Para evitar que el servicio se pare si el SDK del target ≥ 26
 - A partir del SDK 28, hay que pedir permiso para ejecutar el método *startForeground(..)*
 - Es un permiso "normal" y se concede automáticamente

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
```

- Desde SDK 33 también es necesario pedir permiso para que aparezca la notificación. Si no se pide, el servicio se ejecuta sin la notificación presente.



SERVICIOS

○ Service

- Para detener un servicio que se lanzó con *startService(..)*, o *startForegroundService(..)*:
 - *stopService()* desde una actividad

```
Intent i =new Intent(this,MiServicio.class);
stopService(i);
```

- *stopForeground()* y *stopSelf()* desde el propio servicio

```
elservicio.stopForeground(STOP_FOREGROUND_REMOVE);
elservicio.stopSelf();
```



SERVICIOS

○ Service

- En el servicio se pueden sobrescribir algunos métodos:

- *onCreate()*

Al crear la instancia (1 única vez)

- *onStartCommand(Intent intent, int flags, int startID)*

- *onDestroy()*

Cada vez que se haga startService o startForegroundService

Cada vez que se haga stopService



SERVICIOS

○ Service

- ***onStartCommand(Intent intent, int flags, int startID)*** debe devolver un valor entero en forma de constante:
 - START_NOT_STICKY
 - Si el servicio se detiene, no se reinicia
 - START_STICKY
 - Si el servicio se detiene, se ejecuta de nuevo ***onStartCommand*** con los parámetros null (Dirigido a Media Players)
 - START_REDELIVER_INTENT
 - Si el servicio se detiene, se ejecuta de nuevo ***onStartCommand*** con los parámetros de la última llamada (Por ejemplo, una descarga)

Esta constante indica qué hacer con el servicio si es destruido



SERVICIOS

○ Service

- Si queremos que una actividad pueda acceder a lo que está ocurriendo en un servicio hay que enlazarlos (bind)
- Se puede enlazar una actividad a un servicio que ya esté en marcha o si no, se pone en marcha automáticamente al crear el enlace
- El ciclo de vida de la ejecución de un servicio sin enlazar y uno enlazado es distinto

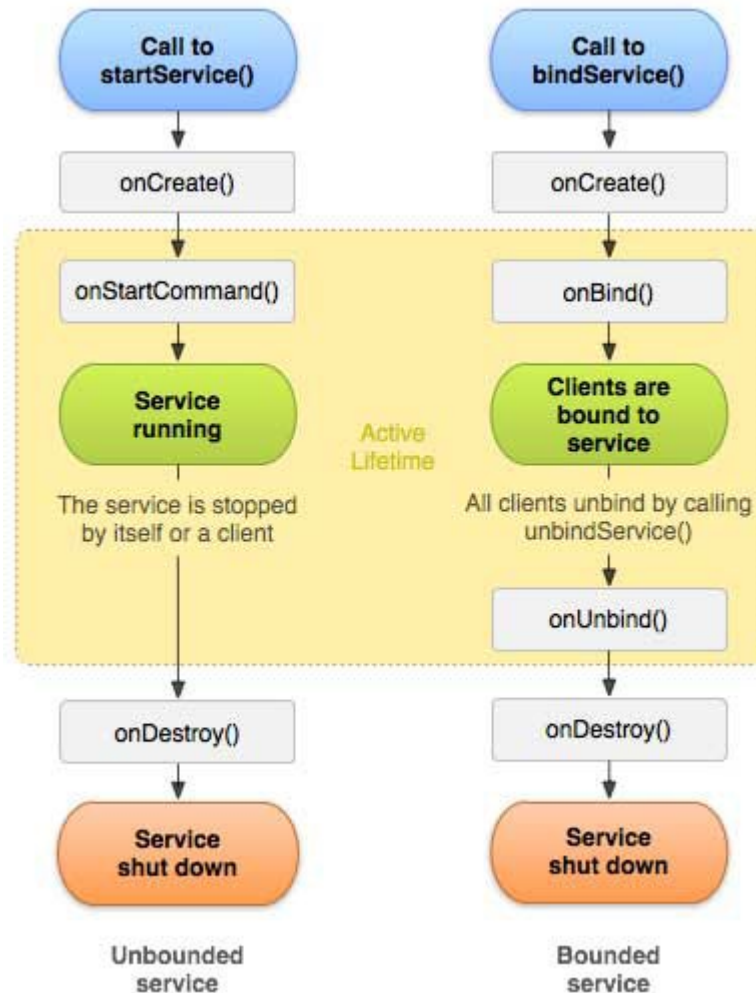


SERVICIOS

- Service
 - Ciclo de vida:

startService()

bindService()



SERVICIOS

○ Service

- En el servicio se pueden sobrescribir algunos métodos:

- `onCreate()`

Al crear la instancia (1 única vez)

- `onStartCommand(Intent intent, int flags, int startID)`

- `onDestroy()`

Cada vez que se haga `startService` o `startForegroundService`

- `onBind(Intent intent)`

Cada vez que se haga `stopService`

Al vincular una actividad al servicio



SERVICIOS

○ Service

- El **Service** puede implementar métodos, de forma que se pueda acceder a ellos desde la actividad.
 - Hay que crear en el servicio una clase que extienda de la clase **Binder** y que tenga un método que devuelva la instancia del servicio

```
public class miBinder extends Binder{  
    MiServicio obtenerServicio() {  
        return MiServicio.this;  
    }  
}
```

La clase del servicio



SERVICIOS

○ Service

- El **Service** puede implementar métodos, de forma que se pueda acceder a ellos desde la actividad.
 - Hay que crear en el servicio una variable de tipo **IBinder** (Interfaz de **Binder**), que contenga la instancia del servicio y devolver esa variable en el método **onBind(..)**

```
public class MiServicio extends Service {  
  
    private final IBinder elBinder= new miBinder();  
  
    @Override  
    public IBinder onBind(Intent intent) {  
        return elBinder;  
    }  
  
    ...  
}
```



SERVICIOS

○ Service

- El **Service** puede implementar métodos, de forma que se pueda acceder a ellos desde la actividad.
 - En la actividad hay que obtener la referencia al servicio haciendo uso de la clase **ServiceConnection**

```
private MiServicio elservicio;  
private ServiceConnection laconexion= new ServiceConnection() {  
    @Override  
    public void onServiceConnected(ComponentName name, IBinder service) {  
        elservicio= ((MiServicio.miBinder) service).obtenServicio();  
    }  
  
    @Override  
    public void onServiceDisconnected(ComponentName name) {  
        elservicio=null;  
    }  
};
```



SERVICIOS

○ Service

- El **Service** puede implementar métodos, de forma que se pueda acceder a ellos desde la actividad.
 - Para unir y desunir la actividad al servicio se usa también la clase **ServiceConnection**

```
Intent intent = new Intent(this, MiServicio.class);  
bindService(intent, laconexion, Context.BIND_AUTO_CREATE);
```

Ya se puede llamar a los métodos públicos del servicio mediante `elservicio.nombremétodo`

Si el servicio está en marcha, se une. Si el servicio está parado, lo crea y se une

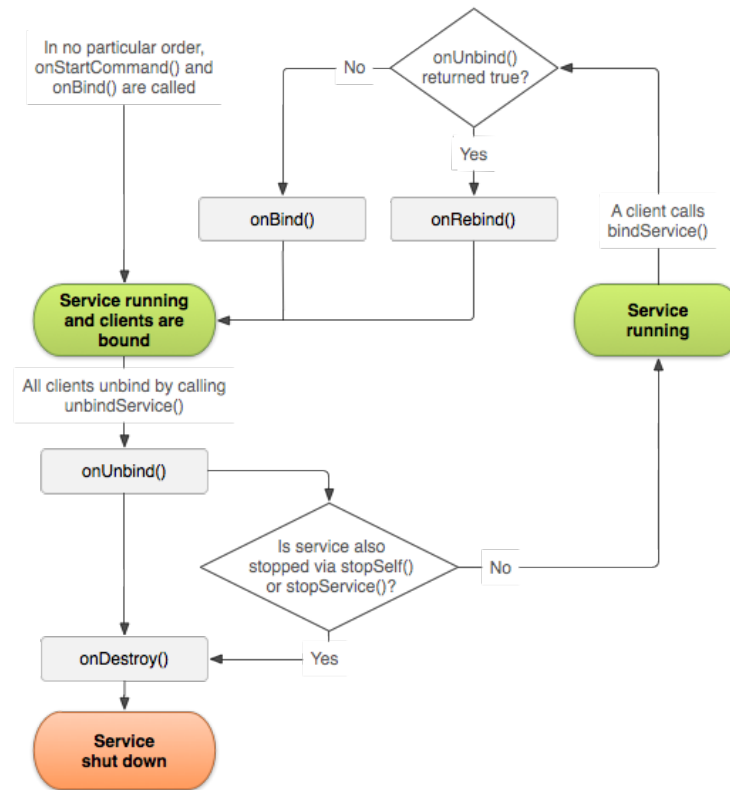
```
unbindService(laconexion);
```

- Un servicio enlazado se detendrá al destruir el último de los enlaces



SERVICIOS

○ Ciclo de vida del servicio enlazado



SERVICIOS

○ Ejercicio 1

- Implementad una aplicación que tenga un botón que ponga en marcha un servicio
- El servicio lo único que va a hacer es controlar el tiempo transcurrido desde que se puso en marcha
 - Para calcular diferencias de tiempo entre dos variables de tipo Date

En milisegundos

```
Date fechaactual=new Date();  
long diferencia=fechaactual.getTime() - fechacomienzo.getTime();
```

- Incluid en la aplicación un botón que al pulsarlo saque un mensaje (un toast, un diálogo, una etiqueta, etc.) con el tiempo transcurrido desde que se puso en marcha el servicio
- Incluid un botón que pare el servicio



BROADCAST

- Un Broadcast es un aviso o evento
- Puede ser un aviso general del sistema

```
<action android:name="android.intent.action.BOOT_COMPLETED"></action>
```

- Puede ser un aviso generado desde una aplicación
 - Implícito

```
Intent intentBC = new Intent();  
intentBC.setAction("avisoPersonalizado");  
sendBroadcast(intentBC);
```

No se dice quién debe recogerlo

- Explícito

```
Intent intentBC = new Intent(this, ElReceiver.class);  
intentBC.setAction("avisoPersonalizado");  
sendBroadcast(intentBC);
```

Se dice quién debe recogerlo



BROADCAST

○ Broadcast Receiver

- Reciben avisos de ***broadcast***. Pueden despertar aplicaciones.
- Extienden la clase ***BroadcastReceiver***

```
public class ElReceiver extends BroadcastReceiver {  
    ...  
}
```

Puede ser una clase independiente o definirse dentro de una actividad. Si se hace dentro de la actividad, debe ser static

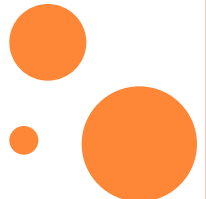


BROADCAST

- Broadcast Receiver estáticos
 - Deben ir declarados en el *manifest*, dentro del elemento `<application>`
 - Mediante un *intent-filter* le diremos a qué avisos tiene que reaccionar

.MiActividad\$ElReceiver si se ha definido de manera interna a la actividad

```
<receiver android:name=".ElReceiver">  
  <intent-filter>  
    <action android:name="android.intent.action.BOOT_COMPLETED"></action>  
  </intent-filter>  
</receiver>
```

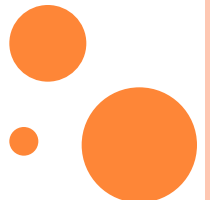


BROADCAST

- Broadcast Receiver

- A partir del API 26, está limitado qué avisos se pueden declarar en el manifiesto
 - Los explícitos
 - Algunos de los generados por el sistema

<https://developer.android.com/guide/components/broadcast-exceptions.html>



BROADCAST

- Broadcast Receiver dinámicos
 - También se puede registrar un **Broadcast Receiver** mediante código, usando el método **registerReceiver()** de la clase **Activity**:

```
IntentFilter filter = new IntentFilter("avisoPersonalizado");  
ElReceiver receiver = new ElReceiver();  
registerReceiver(receiver, filter);
```

Forma de evitar la
restricción en API >= 26

- Y se puede eliminar el registro:

```
unregisterReceiver(receiver);
```



BROADCAST

○ Broadcast Receiver

- Cuando se recibe un aviso de mensaje de *broadcast*, se ejecuta el método *onReceive (Context, intent)*

```
public class ElReceiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        ...  
    }  
}
```

Podemos saber qué aviso estamos recibiendo a través del String que devuelve `intent.getAction()`



BROADCAST

- Broadcast Receiver

- Algunos avisos de **Broadcast** necesitan permisos específicos en el AndroidManifest.xml

```
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
```



BROADCAST

○ Ejercicio 2

- Implementad una aplicación que lance una notificación cada vez que se pase el dispositivo a modo Avión.

```
<intent-filter>  
    <action android:name="android.intent.action.AIRPLANE_MODE" />  
</intent-filter>
```

